

ResQGrid

Complete Phase-Wise Implementation Plan (v3 — BLE Now Core, Not Optional)

12 Phases • Research-Gap Mapping • Timeline • Team Split • Cost Breakdown

Table of Contents

- 1. How to Read This Plan
- 2. What Changed in This Version (v3)
- 3. Phase 1 — Planning & Setup
- 4. Phase 2 — Database Design
- 5. Phase 3 — Backend Core APIs
- 6. Phase 4 — CADERF (Evidence Reconciliation Engine)
- 7. Phase 5 — Incident Timeline + Local Knowledge Module
- 8. Phase 6 — Citizen App + ResQ Multi-Role App + BLE Offline-Relay (CORE)
- 9. Phase 7 — Admin/Government Dashboard
- 10. Phase 8 — Real-Time Layer (Socket.io)
- 11. Phase 9 — LoRa Hardware Integration (Optional Bonus)
- 12. Phase 10 — Simulation & Evaluation
- 13. Phase 11 — Testing & Polish
- 14. Phase 12 — Documentation & Presentation Prep
- 15. Master Summary — Phase vs Research Gap Mapping
- 16. Timeline Overview
- 17. Team-Split Suggestion
- 18. Cost Breakdown

1. How to Read This Plan

Ye document ResQGrid ko banane ka poora, sequential roadmap hai. Har phase mein: (1) Kya banana hai, (2) Deliverable, aur (3) agar Research-Gap ya Key-Feature cover ho raha ho, to use highlighted box mein point out kiya gaya hai — orange (Research-Gap), green (Key-Feature), blue (Core-Feature, Not Optional).

4 Confirmed Research Gaps: Gap 1 (Evidence Reconciliation), Gap 2 (Unified Data Model), Gap 3 (Persistent Local Knowledge), Gap 4 (Experimental Evaluation).

2. What Changed in This Version (v3)

Zaroori Correction: Pichli version mein BLE (Bluetooth Offline-Relay) aur LoRa dono ko ek saath 'Optional Bonus' bola gaya tha — ye galat clubbing thi.

- BLE Offline-Relay ab 'CORE FEATURE' hai — ye Phase 6 ke andar hi, Citizen/ResQ App banate waqt hi implement hoga. Ye project ka 'Offline-First' naam ka asal matlab hai — bina internet ke bhi SOS phone-se-phone relay ho sake.
- LoRa Hardware Integration akela 'Optional Bonus' rehta hai (ab standalone Phase 9) — kyunki ye extra-hardware-cost aur hardware-risk involve karta hai, jabki BLE sirf software hai aur phone mein already available hardware use karta hai.
- Phase-numbering thodi update hui hai (9A/9B ki jagah ab Phase 6 mein BLE core hai, aur sirf Phase 9 LoRa ke liye hai).

PHASE 1 — Planning & Setup (2-3 Days)

- GitHub repository banao, folder-structure decide karo (server/, resqgrid_app/)
- MongoDB Atlas account + free-cluster setup
- Node.js project init (npm init), Express install
- .env file setup (PORT, MONGO_URI, JWT_SECRET)

Deliverable: Empty project-skeleton, GitHub par pehla commit.

PHASE 2 — Database Design (3-4 Days)

- Mongoose Schemas: EvidenceReport, Incident, User (role-based), LocalKnowledge, ResourceInventory
- Seeder script — dummy villages, camps, hospitals insert karne ke liye

★ **GAP 2 — EvidenceReport schema hi wo 'Unified Evidence Data Model' hai jisme Citizen/Volunteer/Hospital/Drone sabka data ek common structure mein store hota hai.**

Deliverable: Saare models ready, MongoDB mein test-data dikhna chahiye.

PHASE 3 — Backend Core APIs (5-6 Days)

- Authentication (JWT, role-based access — 7 roles)
- CRUD APIs: /api/sos, /api/volunteer-update, /api/hospital-update, /api/ngo-inventory, /api/camp-status, /api/knowledge

Deliverable: Postman se test karke saare basic-APIs kaam karte dikhne chahiye.

PHASE 4 — CADERF — Evidence Reconciliation Engine (7-8 Days)

Ye poore project ka core research-module hai.

- Normalization function, Relationship-detection logic (Supports/Conflicts/Updates/Duplicates/Completes)
- Reconciliation algorithm — multiple reports se ek Unified-Incident banane wala logic

★ **GAP 1 — DISASTER-SPECIFIC EVIDENCE RECONCILIATION — Ye is poori phase ka core hai, tumhare research-paper ka sabse important contribution.**

 **KEY FEATURE — Duplicate-Filtering Bottleneck — Rescue-Team tak pahunchne se pehle saare duplicate-reports filter ho jaate hain. Sirf ek clean, deduplicated 'Unified Incident' Rescue-Team ke dashboard tak pahunchta hai.**

Deliverable: 5 fake conflicting reports do, system se ek clean, deduplicated Incident nikal ke dikhao.

PHASE 5 — Incident Timeline + Local Knowledge Module (3-4 Days)

- Har Incident ke saath timestamped-history save karna
- Local-Knowledge ke 10 categories (Road, Bridge, Shelter, Boat Launch, Water Depth, Village Contact, Local Language, Medical Resource, Livestock Area, Danger Zone)

 **GAP 3 — PERSISTENT LOCAL DISASTER KNOWLEDGE REPRESENTATION — Local-knowledge entries kabhi delete nahi hoti, sirf update hoti hain.**

Deliverable: Ek Incident ki timeline aur ek Village ka Local-Knowledge database mein dikhna chahiye.

PHASE 6 — Citizen App + ResQ Multi-Role App + BLE Offline-Relay (Flutter) (10-12 Days)

Ye phase ab teen cheezein saath-saath cover karta hai — kyunki BLE offline-relay in-app hi build hota hai, alag se nahi.

Part A — Citizen App

- SOS-form (Rescue/Food/Water/Medicine/Medical-Emergency), photo-upload, GPS-capture
- SOS-status tracking screen (Sent → Relayed → Delivered → Acknowledged)
- Nearby Camp/Hospital status view, Local-Knowledge view

Part B — ResQ App (Role-Based, Single Codebase)

- Volunteer / Hospital / NGO / Camp / Rescue-Team roles — apne-apne dashboards
- Backend APIs se connect (http/dio package)

Part C — BLE Offline-Relay Engine (CORE, Built-In From Day 1)

- flutter_blue_plus package integrate karo
- Communication Abstraction Layer: Internet available? → Direct-send. Nahi? → Nearby BLE-device dhoondo → Relay → jab kisi bhi device ko internet mile → Cloud-sync
- Local-Queue (on-device storage) jab tak koi relay-path na mile

 **CORE FEATURE (Not Optional) — BLE Offline-Relay ye 'add-on' nahi hai — ye ResQGrid ke naam ('Offline-First') ka asal matlab hai. Isse pehle 'optional bonus' bola gaya tha, jo galat tha. Ye Citizen App aur ResQ App dono ke andar shuru se hi built-in feature hai — koi phone internet ke bina bhi SOS relay kar sake, yehi is poore project ka core-communication-vision hai.**

Deliverable: Citizen App + ResQ App working, aur ek 2-phone live-demo — internet-off karke SOS ek phone se doosre relay hote dikhao, phir cloud tak sync hote dikhao.

PHASE 7 — Admin / Government Dashboard (React Web) (5-6 Days)

- Leaflet Map integration — SOS/Camp/Hospital markers
- Heatmap layer, Incident-list panel (deduplicated), Resource-status panel

Deliverable: Ek working web-dashboard jo live data dikhata hai.

PHASE 8 — Real-Time Layer (Socket.io) (2-3 Days)

- Backend mein Socket.io setup

- Naya SOS/Incident bante hi dashboard automatically update ho (bina refresh)

Deliverable: Ek phone se SOS bhejo, dashboard turant update hote dikhe.

PHASE 9 — LoRa Hardware Integration (Optional Bonus) (3-4 Days)

- 1x ESP32 + LoRa board kharido (₹1,770-2,500), 865-867 MHz India-band
- Arduino IDE setup, LoRa library, test-code upload
- Node.js backend se serialport integration, Dashboard par 'Received via LoRa' tag

Ye akela hardware-dependent phase optional hai — sirf isliye kyunki naya-hardware kharidna padta hai aur hardware-risk hota hai. BLE (Phase 6 mein already core) is optional-list mein NAHI hai. Sirf Phase 1-8 complete hone ke baad, agar time bache, tabhi LoRa shuru karo.

Deliverable: LoRa-module se ek test-message dashboard tak live pahunchte dikhao.

PHASE 10 — Simulation & Evaluation Module (4-5 Days)

- 1000-report dummy-data generator
- 4 methods implement: Raw-Reports / Majority-Vote / Weighted-Fusion-Baseline / CADERF

★ **GAP 4 — EXPERIMENTAL EVALUATION OF DISASTER EVIDENCE FRAMEWORKS.**

Deliverable: Ek results-table/chart jo directly research-paper mein use hoga.

PHASE 11 — Testing, Polish, Bug-Fixing (4-5 Days)

- End-to-end testing, UI polish, error-handling

PHASE 12 — Documentation + Presentation Prep (3-4 Days)

- README, Presentation PPT, Demo-script practice (BLE-relay scene), Research-paper draft

Master Summary — Phase vs Research Gap Mapping

Phase	Kya Cover Hota Hai	Research Gap / Feature
Phase 1	Setup	—
Phase 2	Database Design	★ Gap 2
Phase 3	Backend APIs	—
Phase 4	CADERF Engine	★ Gap 1 + 🔑 Duplicate-Filtering
Phase 5	Timeline + Knowledge Module	★ Gap 3
Phase 6	Citizen + ResQ App + BLE Relay	🔵 CORE (BLE not optional)
Phase 7	Dashboard	—
Phase 8	Real-Time Layer	—
Phase 9	LoRa Hardware	— (Optional, Demo-value only)
Phase 10	Simulation & Evaluation	★ Gap 4
Phase 11-12	Polish + Documentation	—

Motivation Reminder: 'Policy → Software Translation' koi alag phase/gap nahi hai — ye poore project ka overall justification hai (research-paper Introduction section mein).

Timeline Overview

Phase Group	Approx Days
Setup + Database (Phase 1-2)	~6 din
Backend + Core Research-Engine (Phase 3-5)	~18 din
Citizen+ResQ App + BLE-Relay + Dashboard (Phase 6-8)	~20 din
LoRa Bonus + Evaluation (Phase 9-10)	~9 din
Polish + Presentation (Phase 11-12)	~8 din
Total (4-member team, parallel work)	~61-65 din (~2 mahine)

Team-Split Suggestion (4 Members)

Member	Responsibility
Member 1	Backend + CADERF Engine (Phase 2-5) — sabse research-critical role
Member 2	Citizen App + ResQ Multi-Role App + BLE-Relay (Phase 6, Flutter) — sabse bada core-mobile role
Member 3	Dashboard + GIS (Phase 7-8)
Member 4	LoRa (Phase 9) + Simulation/Evaluation (Phase 10) — research-paper-focused role

Cost Breakdown

Item	Cost
Core software (Flutter, Node, MongoDB Atlas free-tier, GitHub)	₹0
BLE Offline-Relay (Flutter, core-feature, no extra build-step)	₹0
Twilio SMS (optional)	Free-trial credits, phir paid
LoRa Hardware (1 device, optional)	₹1,770 – ₹2,500
Google Play Developer Account (optional, not needed for demo)	\$25 one-time (skip-able)
Total Minimum Realistic Budget	₹0 (core, BLE-included) + ₹2,000-2,500 (agar LoRa bhi karna ho)